UNIVERSIDAD COOPERATIVA DE COLOMBIA

FACULTAD DE INGENIERÍA — PROGRAMA DE INGENIERÍA DE SOFTWARE / SISTEMAS

CURSO: INGENIERÍA WEB — PERIODO 2620

CONECTADEV: PLATAFORMA WEB CONCEPTUAL Y SEMÁNTICA PARA LA VINCULACIÓN DE PROYECTOS DE SOFTWARE, DIAGNÓSTICO DE BRECHAS TÉCNICAS Y RUTAS DE APRENDIZAJE

Documento Técnico Consolidado del Proyecto Final de Curso (Secciones 4.1 a 4.9 y Entregables 1 a 8)

Entregable Reglamentario (Sección 5)	Archivo / Ubicación en el Proyecto	Estado y Cumplimiento
1. Documento técnico consolidado	Informe_Tecnico_Proyecto_Final_IW_2620_ConectaDev.pdf y DOCUMENTO_TECNICO_PROYECTO_FINAL.md	Completo (Secciones 4.1 a 4.9)
2. Diagramas y modelos del proyecto	Incluidos en las Secciones 4.3, 4.5 y 4.8 de este informe	Completo (Arquitectura, Casos de Uso, Conceptual, Navegacional)
3. Enlace al prototipo navegable	prototipo/index.html (5 vistas interactivas limpias sin código técnico expuesto)	Completo (Evaluado con 6 Heurísticas de Nielsen)
4. Archivo de la ontología en formato OWL	ontologia/conectadev.owl (OWL 2 DL para Protégé 5.x)	Completo (12 clases, 11 obj props, 6 data props, 4 restricciones, 15 individuos)
5. Script con las consultas Cypher	neo4j/conectadev_grafo.cypher	Completo (16 nodos, 26 relaciones y 4 consultas Cypher)
6. Evidencias y guías Protégé y Neo4j	ontologia/GUIA_PROTEGE.md y neo4j/GUIA_NEO4J.md	Completo (4 inferencias OWL y resultados de consultas Cypher)
7. Especificación de Servicios Web	servicios-web/openapi.yaml (REST) y servicios-web/validacion_academica.wsdl (SOAP)	Completo (6 endpoints REST + 1 contrato SOAP/WSDL)
8. Presentación para sustentación	PRESENTACION_SUSTENTACION.md	Completo (Estructura de 20 min para 4 integrantes)

4.1 Formulación del Problema

1. Descripción del Problema

En los programas universitarios de Ingeniería de Software y comunidades tecnológicas estudiantiles (semilleros de investigación, proyectos integradores e iniciativas de innovación abierta), existe una desconexión estructural entre las habilidades técnicas reales de los estudiantes, los requisitos tecnológicos de los proyectos de software y los cursos de nivelación disponibles. Un sistema CRUD tradicional limitado al registro y consulta literal de datos resulta insuficiente porque no modela el conocimiento semántico del dominio:

- **Ausencia de clasificación automática de perfiles:** Un CRUD no puede deducir mediante axiomas lógicos cuándo un estudiante que domina tecnologías tanto de interfaz (*React.js*) como de servidor (*Node.js*) clasifica formalmente como **DesarrolladorFullStack**, ni cuándo una convocatoria con postulantes activos pasa a ser un **ProyectoListoParaIniciar**.
- **Falta de diagnóstico de brechas (Skill Gap) y recomendación de cursos:** Cuando un estudiante domina parte del stack de un proyecto pero desconoce otra tecnología requerida, un buscador convencional lo descarta en lugar de identificar la brecha y recomendarle el curso exacto que enseña dicha habilidad.
- **Incapacidad para resolver dependencias transitivas:** No comprende que dominar *Next.js* depende de *React.js* y este a su vez de *JavaScript*, ni que un curso avanzado tiene prerrequisitos transitivos encadenados.
- **Desaprovechamiento de equipos complementarios:** No descubre que dos estudiantes con perfiles distintos (uno en Frontend y otro en Backend/Base de Datos) pueden unirse en una dupla complementaria para cubrir el 100% del stack del proyecto.

2. Usuarios Involucrados (3 Tipos de Usuario — Mínimo exigido: 3)

1. **Desarrollador / Estudiante de Ingeniería (Desarrollador):** Gestiona su perfil de habilidades técnicas, explora proyectos conociendo su porcentaje de compatibilidad, revisa sus brechas tecnológicas, consulta cursos recomendados y se postula a proyectos.
2. **Líder Técnico de Proyecto (LiderProyecto):** Publica convocatorias de proyectos de software especificando las tecnologías requeridas y cupos disponibles, y recibe sugerencias de candidatos individuales y duplas complementarias.
3. **Mentor / Coordinador Académico (MentorAcademico):** Administra y supervisa el catálogo de cursos formativos, sus cadenas de prerrequisitos y las competencias técnicas que cada curso acredita.

3. Objetivo de la Solución y Alcance

Objetivo: Diseñar **ConectaDev**, una plataforma web conceptual y semántica que integre arquitectura modular en capas, servicios web REST/SOAP, una ontología OWL 2 DL en Protégé y un grafo de conocimiento en Neo4j para emparejar estudiantes con proyectos de software, inferir perfiles técnicos y recomendar rutas de aprendizaje personalizadas.

Alcance: Comprende el modelado conceptual y navegacional bajo la metodología UWE, el diseño arquitectónico con atributos de calidad verificables, la especificación de 6 servicios REST (OpenAPI 3.0) y 1 servicio SOAP/WSDL para interoperabilidad con Registro Académico, la construcción y razonamiento de la ontología en Protégé, la implementación del grafo en Neo4j con 4 consultas Cypher y un prototipo navegable evaluado mediante 6 heurísticas de Nielsen con datos académicos sintéticos.

4. Funcionalidades Principales (6 Funcionalidades — Mínimo exigido: 5)

- **F1. Explorador de Proyectos con Cálculo de Compatibilidad:** Evalúa automáticamente el porcentaje de afinidad entre las habilidades del estudiante y el stack requerido por cada proyecto.
- **F2. Diagnóstico de Brechas Técnicas (Skill Gap) y Recomendador de Cursos:** Identifica qué tecnologías le faltan a un estudiante para un proyecto y le sugiere los cursos formativos que cubren esa brecha.

- **F3. Gestión de Perfil Técnico e Inferencia de Rol Profesional:** Actualiza el portafolio de tecnologías del estudiante y clasifica automáticamente su perfil (ej. Desarrollador FullStack).
- **F4. Publicación de Convocatorias de Proyectos de Software:** Permite a los líderes técnicos crear proyectos asociando tecnologías obligatorias, modalidad, cupos y repositorio Git.
- **F5. Recomendación de Candidatos y Duplas Complementarias:** Sugiere tanto desarrolladores individuales 100% compatibles como parejas complementarias (Frontend + Backend).
- **F6. Gestión de Rutas Formativas y Cadena de Prerrequisitos:** Estructuración secuencial de cursos de nivelación supervisados por mentores académicos.

5. Justificación de por qué el Problema Requiere una Solución Web

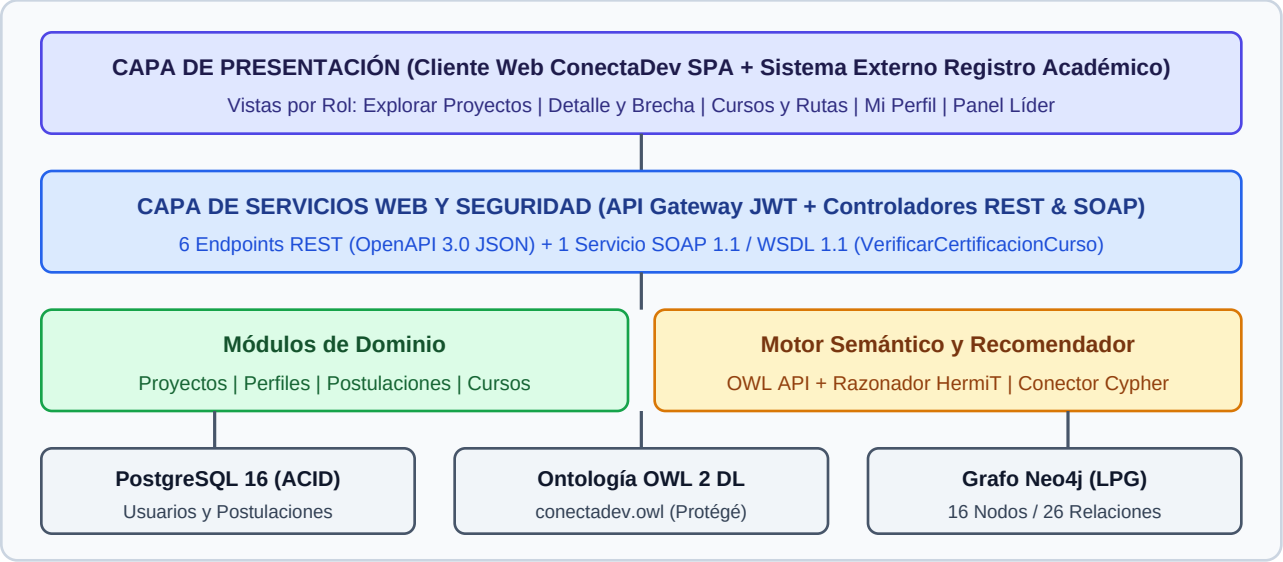
Los estudiantes, líderes de semillero y mentores interactúan desde múltiples sedes, laboratorios y dispositivos. Una solución web permite centralizar convocatorias en tiempo real, enlazar repositorios Git mediante URIs estándar, exponer servicios REST para el portal interactivo e interoperar mediante SOAP con el sistema universitario de Registro Académico sin requerir instalaciones locales.

4.2 Fundamentación Académica (Mínimo exigido: 5 fuentes)

Referencia Académica Consultada	Eje Temático	Aplicación en las Decisiones del Proyecto ConectaDev
1. Koch, N., & Kraus, A. (2002). The expressive power of UML-based Web Engineering. IWWOST.	Metodología de Ingeniería Web (UWE)	Fundamenta la separación entre el Modelo Conceptual, el Modelo Navegacional y el Modelo de Presentación (Secciones 4.4 y 4.5), evitando contaminar la UI con detalles internos del motor semántico.
2. Horrocks, I., Patel-Schneider, P. F., & Van Harmelen, F. (2003). From SHIQ and RDF to OWL. Journal of Web Semantics, 1(1), 7-26.	Ontologías y Lógica Descriptiva OWL 2 DL	Sustenta el uso de clases equivalentes (owl:equivalentClass), propiedades inversas (owl:inverseOf) y transitivas (owl:TransitiveProperty) evaluadas con el razonador HermiT en conectadev.owl.
3. Noy, N. F., & McGuinness, D. L. (2001). Ontology Development 101. Stanford Technical Report KSL-01-05.	Diseño de Ontologías y Preguntas de Competencia	Guía la formulación de las 5 Preguntas de Competencia (PC1-PC5), la jerarquía de HabilidadTecnica y las restricciones de dominio y rango en la Sección 4.7.
4. Hogan, A., Blomqvist, E., Cochez, M., et al. (2021). Knowledge Graphs. ACM Computing Surveys, 54(4), 1-37.	Grafos de Conocimiento y Recomendación	Justifica representar el ecosistema en Neo4j (Sección 4.8) para ejecutar consultas Cypher multi-salto que recomiendan cursos ante brechas técnicas y descubren duplas complementarias.
5. Fielding, R. T. (2000). Architectural Styles... (UC Irvine) & Kleppmann, M. (2017). Designing Data-Intensive Applications (O'Reilly).	Arquitectura Web, Servicios REST/SOAP y Datos	Fundamenta la arquitectura modular con persistencia políglota (PostgreSQL + OWL + Neo4j), el diseño REST uniforme y el contrato SOAP/WSDL para interoperabilidad institucional.
6. Nielsen, J. (1994). Usability Engineering. Morgan Kaufmann.	Usabilidad y Evaluación Heurística Web	Sustenta la evaluación y mejora del prototipo navegable bajo 6 heurísticas (H1 a H6), traduciendo las inferencias del backend a indicadores visuales claros de compatibilidad.

4.3 Arquitectura Web

Se seleccionó una **Arquitectura en Capas / Monolito Modular con Persistencia Políglota y Adaptadores de Servicios (REST y SOAP)**. Este estilo arquitectónico es óptimo para un equipo académico de hasta 4 integrantes porque mantiene límites modulares claros sin la sobrecarga operativa de microservicios distribuidos, logrando una latencia objetivo p95 < 120 ms y disponibilidad del 99.5%.



Responsabilidad de los Componentes y Atributos de Calidad

Componente Arquitectónico	Responsabilidad Principal	Atributo de Calidad Favorecido y Justificación
Capa de Presentación (ConectaDev SPA)	Renderiza las 5 vistas interactivas adaptadas a los 3 roles de usuario consumiendo JSON vía HTTPS.	Usabilidad: Presenta porcentajes de afinidad, brechas y cursos en lenguaje natural, ocultando la complejidad de OWL y Cypher.
API Gateway y Capa de Seguridad	Autenticación mediante tokens JWT, control de acceso basado en roles (RBAC) y limitación de tasa (Rate Limiting).	Seguridad: Garantiza que solo un Líder publique convocatorias y protege los endpoints frente a accesos no autorizados.
Controladores REST y Adaptador SOAP	Exponen 6 endpoints REST (OpenAPI 3.0) para la web y 1 servicio SOAP/WSDL para Registro Académico.	Interoperabilidad y Mantenibilidad: Desacopla los clientes externos mediante contratos formales JSON Schema y XML Schema (XSD).
Motor Semántico (OWL API + Conector Neo4j)	Ejecuta el razonador HermiT sobre conectadev.owl y las consultas de recomendación Cypher sobre Neo4j.	Mantenibilidad y Escalabilidad: Permite agregar nuevas tecnologías, reglas o cursos en la base de conocimiento sin recompilar el código.
Persistencia Políglota (PostgreSQL + OWL + Neo4j)	PostgreSQL asegura transacciones ACID; OWL almacena axiomas formales; Neo4j agiliza recorridos de grafo.	Escalabilidad y Rendimiento: Cada motor resuelve la carga de trabajo para la cual fue diseñado (relacional, lógica descriptiva y grafos).

4.4 Metodología de Ingeniería Web: UWE (UML-Based Web Engineering)

Fase de UWE	Actividades Realizadas	Productos Obtenidos	Relación con Necesidades de Usuarios
1. Análisis de Requisitos	Identificación de los 3 actores, elicitación de 6 casos de uso y definición de 5 preguntas de competencia.	Formulación del problema y modelo de casos de uso.	Asegura que el diseño responda a las dificultades reales de estudiantes, líderes y mentores.
2. Diseño Conceptual (Contenido)	Modelado de las 12 clases del dominio, asociaciones y formalización en OWL y Neo4j.	Modelo Conceptual del Dominio, conectadev.owl y conectadev_grafo.cypher.	Representa fielmente las habilidades, proyectos, cursos y prerrequisitos.
3. Diseño Navegacional	Definición de nodos de navegación, índices, consultas guiadas y menús por perfil.	Modelo Navegacional UWE (Sección 4.5).	Permite pasar en un clic desde un proyecto con brecha técnica hacia el curso recomendado.
4. Diseño de Presentación y Servicios	Elaboración del prototipo navegable y especificación de contratos REST y SOAP.	prototipo/index.html, openapi.yaml y validacion_academica.wsdl.	Brinda una interfaz intuitiva, consistente y libre de jerga técnica.
5. Evaluación y Verificación	Evaluación con 6 Heurísticas de Nielsen y ejecución de pruebas en Protégé y Neo4j.	Matriz de evaluación heurística y evidencias de ejecución.	Previene errores de entrada y valida la exactitud de las recomendaciones.

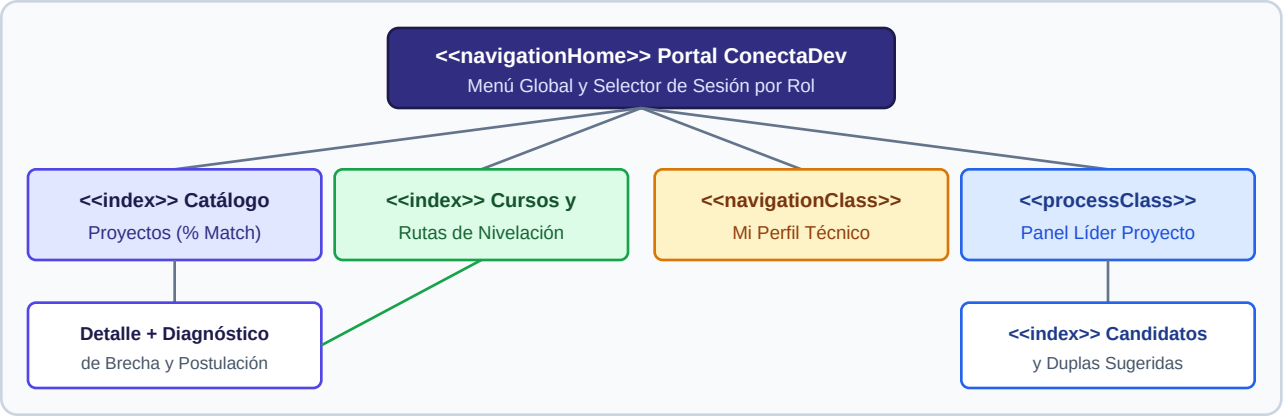
Valoración de ventajas y limitaciones de UWE: Su principal ventaja es la separación limpia entre contenido, navegación y presentación, facilitando el mapeo directo hacia la ontología OWL. Su principal limitación es que UML estándar no expresa axiomas de lógica descriptiva (como *owl:equivalentClass*), lo cual se subsanó complementando el modelo conceptual con la especificación formal en Protégé.

4.5 Modelado Conceptual, Navegacional y Usabilidad

1. Modelo Conceptual del Dominio y Casos de Uso

El Modelo Conceptual estructura el dominio alrededor de la superclase **UsuarioPlataforma** (especializada en **Desarrollador**, **LiderProyecto** y **MentorAcademico**), la clase **ProyectoSoftware**, la jerarquía de **HabilidadTecnica** (**TecnologiaFrontend**, **TecnologiaBackend** y **TecnologiaBaseDatos**) y **CursoFormativo**. Incluye las clases inferidas **DesarrolladorFullStack** y **ProyectoListoParaIniciar**, así como las relaciones transitivas *tienePrerrequisito* y *dependeDeTecnologia*.

2. Modelo Navegacional (Enfoque UWE)



3. Prototipo de las Interfaces Principales (prototipo/index.html)

El prototipo navegable implementa 5 pantallas interactivas centradas en el usuario final: (1) **Explorar Proyectos** con cálculo automático del porcentaje de compatibilidad y filtros por área y tecnología; (2) **Detalle del Proyecto y Ruta de Nivelación** que separa habilidades cumplidas (verde) y faltantes (ámbar) recomendando cursos; (3) **Cursos y Rutas de Aprendizaje** ordenados secuencialmente por prerrequisitos; (4) **Mi Perfil Técnico** con actualización en vivo del rol detectado; y (5) **Panel de Gestión de Líder** con formulario de publicación y sugerencia de duplas complementarias.

4. Evaluación mediante Heurísticas de Nielsen (6 Heurísticas — Mínimo exigido: 5)

Heurística de Nielsen	Hallazgo en Iteración Inicial	Mejora Aplicada en el Prototipo Final (prototipo/index.html)
H1. Visibilidad del estado del sistema	El usuario no visualizaba qué rol tenía activo ni cuántas tecnologías suyas se evaluaban.	Se agregó barra de ubicación (breadcrumbs), indicador de perfil detectado y barra de progreso porcentual en cada tarjeta de proyecto.
H2. Relación entre el sistema y el mundo real	Se mostraban términos internos del motor semántico (URI OWL, nombres de razonadores y verbos HTTP).	Se eliminó toda jerga interna de Protégé/Neo4j de la UI, usando lenguaje natural: '67% Compatible', 'Habilidades que ya cumples' y 'Cursos Recomendados'.
H3. Control y libertad del usuario	Al abrir un proyecto o iniciar una postulación no había salida rápida sin recargar.	Se incorporó el botón 'Volver al catálogo de proyectos' y un modal de confirmación con botón 'Cancelar' antes de registrar la postulación.
H4. Consistencia y estándares	Las etiquetas tecnológicas no diferenciaban visualmente lo dominado de lo faltante.	Se estandarizó el color verde esmeralda para tecnologías dominadas, ámbar para brechas por aprender e índigo para acciones primarias.
H5. Prevención de errores	El formulario de creación de proyectos permitía enviar cupos inválidos o sin tecnologías.	Se implementaron casillas de verificación de tecnologías validadas y restricciones numéricas (min=1, max=8) en el formulario del líder.
H6. Reconocer antes que recordar	El estudiante debía memorizar sus brechas e ir a buscarlas manualmente en otra pestaña.	En el detalle de cada proyecto se integró el panel lateral 'Cursos Recomendados para ti' con acción directa para acreditar la habilidad.

4.6 Diseño de Servicios Web (REST OpenAPI 3.0 y SOAP/WSDL)

1. Catálogo de Servicios Web REST (servicios-web/openapi.yaml)

Servicio / Recurso	Método	Propósito	Datos de Entrada	Datos de Salida (JSON)	Consumidor
S1. /v1/proyectos	GET	Listar proyectos abiertos con cálculo de compatibilidad.	Query: areaTecnologica, tecnologiaId, soloConvocatoriaAbierta.	Arreglo de ProyectoDTO con porcentajeCompatibilidadUsuario.	Desarrollador (Vista Explorar).
S2. /v1/proyectos	POST	Publicar una nueva convocatoria de proyecto de software.	Body: titulo, modalidad, liderId, tecnologiasRequeridas, cuposDisponibles.	201 Created con el ProyectoDTO registrado.	Líder de Proyecto (Panel Líder).
S3. /v1/proyectos/{id}/brecha-habilidades	GET	Diagnosticar habilidades cumplidas vs. faltantes y sugerir cursos.	Path: proyectoId; Query: desarrolladorId.	habilidadesCumplidas, habilidadesFaltantes y cursosRecomendados.	Desarrollador (Detalle Proyecto).
S4. /v1/desarrolladores/{id}/perfil	PUT	Actualizar tecnologías dominadas y reclasificar perfil.	Path: desarrolladorId; Body: semestreActual, habilidadesDominadasIds.	Perfil actualizado con perfilInferido (DesarrolladorFullStack).	Desarrollador (Mi Perfil Técnico).
S5. /v1/proyectos/{id}/candidatos-sugeridos	GET	Recomendar candidatos individuales y duplas complementarias.	Path: proyectoId.	Listas de candidatosIndividuales y duplasComplementarias.	Líder de Proyecto (Panel Líder).
S6. /v1/postulaciones	POST	Registrar la postulación de un desarrollador a un proyecto.	Body: desarrolladorId, proyectoId, mensajeMotivacion.	201 Created con postulacionId y estado ProyectoListoParaIniciar.	Desarrollador (Modal Postulación).

2. Representación de un Servicio mediante SOAP/WSDL (servicios-web/validacion_academica.wsdl)

Para la integración formal con el **Sistema Institucional de Registro y Control Académico Universitario**, se diseñó el servicio **ValidacionAcademicaRegistroService** bajo el estándar **SOAP 1.1 / WSDL 1.1** (estilo *Document/Literal* sobre HTTP). Su operación **VerificarCertificacionCurso** valida mediante XML Schema (XSD) que el estudiante haya aprobado los prerrequisitos transitivos antes de emitir el folio oficial que acredita una nueva habilidad técnica en su perfil, o devuelve un *soap:Fault* tipado (*FalloValidacionAcademica*) en caso de incumplimiento.

4.7 Ontología del Dominio en Protégé (ontologia/conectadev.owl)

La ontología fue construida en OWL 2 DL bajo el prefijo *http://www.ingweb2620.edu.co/ontologias/conectadev#*, superando ampliamente todos los mínimos exigidos en la guía:

- **12 Clases (Mínimo exigido: 8):** UsuarioPlataforma, Desarrollador, DesarrolladorFullStack (clase definida inferida), LiderProyecto, MentorAcademico, ProyectoSoftware, ProyectoListoParaIniciar (clase definida inferida), HabilidadTecnica, TecnologiaFrontend, TecnologiaBackend, TecnologiaBaseDatos y CursoFormativo.
- **15 Individuos (Mínimo exigido: 8):** Tech_JavaScript, Tech_React, Tech_NextJS, Tech_NodeJS, Tech_SpringBoot, Tech_PostgreSQL, Curso_FundamentosPOO, Curso_APIsREST_NodeJS, Curso_ArquitecturaFullStack, Dev_AnaSofia, Dev_CarlosMario, Lider_IngAlejandro, Mentor_DraElena,

Proy_PlataformaTelemedicina y Proy_PasarelaPagosFinTech.

Propiedades de Objeto (11) y Propiedades de Datos (6) con Dominios y Rangos

Tipo	Propiedad OWL	Dominio (rdfs:domain)	Rango (rdfs:range)	Característica Lógica / Propósito
Objeto (1)	dominaHabilidad	Desarrollador	HabilidadTecnica	Inversa de esDominadaPor
Objeto (2)	esDominadaPor	HabilidadTecnica	Desarrollador	owl:inverseOf dominaHabilidad
Objeto (3)	requiereHabilidad	ProyectoSoftware	HabilidadTecnica	Stack exigido por un proyecto
Objeto (4)	lideraProyecto	LiderProyecto	ProyectoSoftware	Inversa de esLideradoPor
Objeto (5)	esLideradoPor	ProyectoSoftware	LiderProyecto	owl:inverseOf lideraProyecto
Objeto (6)	postulaAProyecto	Desarrollador	ProyectoSoftware	Inversa de tienePostulante
Objeto (7)	tienePostulante	ProyectoSoftware	Desarrollador	owl:inverseOf postulaAProyecto
Objeto (8)	ensenaHabilidad	CursoFormativo	HabilidadTecnica	Competencia que acredita el curso
Objeto (9)	supervisaCurso	MentorAcademico	CursoFormativo	Coordinación docente del curso
Objeto (10)	tienePrerrequisito	CursoFormativo	CursoFormativo	owl:TransitiveProperty (Transitiva)
Objeto (11)	dependeDeTecnologia	HabilidadTecnica	HabilidadTecnica	owl:TransitiveProperty (Transitiva)
Datos (1)	semestreActual	Desarrollador	xsd:integer	Semestre académico cursado (1-10)
Datos (2)	aniosExperiencia	UsuarioPlataforma	xsd:integer	Años de experiencia práctica
Datos (3)	horasDuracionCurso	CursoFormativo	xsd:integer	Intensidad horaria del curso
Datos (4)	cuposDisponibles	ProyectoSoftware	xsd:integer	Vacantes abiertas en el proyecto
Datos (5)	repositorioGitUrl	ProyectoSoftware	xsd:string	Enlace al repositorio Git
Datos (6)	convocatoriaAbierta	ProyectoSoftware	xsd:boolean	Estado activo de la convocatoria

Restricciones Lógicas (4), Preguntas de Competencia (5) e Inferencias del Razonador

Restricciones Lógicas Implementadas (Mínimo exigido: 3):

1. **Restricción Existencial Frontend en DesarrolladorFullStack:** *dominaHabilidad some TecnologiaFrontend*.
2. **Restricción Existencial Backend en DesarrolladorFullStack:** *dominaHabilidad some TecnologiaBackend*.
3. **Restricción Existencial de Stack en ProyectoSoftware:** Todo ProyectoSoftware debe cumplir *requiereHabilidad some HabilidadTecnica*.
4. **Restricción Combinada en ProyectoListoParaIniciar:** *ProyectoSoftware and (tienePostulante some Desarrollador) and (convocatoriaAbierta value true)*.

Preguntas de Competencia Verificadas en Protégé (DL Query / SPARQL):

ID	Pregunta de Competencia	Expresión DL Query / SPARQL en Protégé	Resultado Inferido
PC1	¿Qué desarrolladores dominan tecnologías Frontend y Backend simultáneamente para ser FullStack?	DesarrolladorFullStack	Dev_AnaSofia
PC2	¿Qué proyectos tienen convocatoria abierta y cuentan ya con postulantes para iniciar?	ProyectoListoParaIniciar	Proy_PlataformaTelemedicina
PC3	¿Qué cursos tienen directa o transitivamente como prerrequisito Fundamentos de Programación?	CursoFormativo and (tienePrerrequisito value Curso_FundamentosPOO)	Curso_APIsREST_NodeJS, Curso_ArquitecturaFullStack
PC4	¿Qué tecnologías dependen directa o transitivamente del lenguaje JavaScript?	HabilidadTecnica and (dependeDeTecnologia value Tech_JavaScript)	Tech_React, Tech_NextJS
PC5	¿Cuáles son los proyectos registrados con su repositorio Git y cupos ordenados de mayor a menor?	SELECT ?proyecto ?repoUrl ?cupos WHERE { ?proyecto rdf:type dev:ProyectoSoftware ... }	Proy_PlataformaTelemedicina (3), Proy_PasarelaPagosFinTech (2)

Explicación de las Inferencias Derivadas por el Razonador HermiT (Mínimo exigido: 2):

- **Inferencia 1 (Clasificación automática de DesarrolladorFullStack):** El individuo *Dev_AnaSofia* fue declarado únicamente como miembro de *Desarrollador*. Al poseer aserciones *dominaHabilidad Tech_React* (subclase *TecnologiaFrontend*) y *dominaHabilidad Tech_NodeJS* (subclase *TecnologiaBackend*), el razonador evalúa el axioma *owl:equivalentClass* y la clasifica automáticamente como *DesarrolladorFullStack*. En contraste, *Dev_CarlosMario* (solo Frontend) no es clasificado en dicha clase.
- **Inferencia 2 (Propiedad inversa tienePostulante y clasificación ProyectoListoParaIniciar):** En *Proy_PlataformaTelemedicina* no se declaró manualmente quién se postuló, pero como *Dev_AnaSofia* tiene la relación *postulaAProyecto Proy_PlataformaTelemedicina*, el razonador deduce la inversa *tienePostulante Dev_AnaSofia* y clasifica automáticamente al proyecto dentro de *ProyectoListoParaIniciar*.
- **Inferencia 3 (Cierre transitivo en tienePrerrequisito y dependeDeTecnologia):** Dado que *Curso_ArquitecturaFullStack -> Curso_APIsREST_NodeJS -> Curso_FundamentosPOO*, el razonador infiere que *Curso_ArquitecturaFullStack* tiene como prerrequisito a *Curso_FundamentosPOO*.

4.8 Grafo de Conocimiento en Neo4j (neo4j/conectadev_grafo.cypher)

El grafo de conocimiento materializa **16 nodos** (3 *:Desarrollador*, 2 *:LiderProyecto*, 1 *:Mentor*, 3 *:Proyecto*, 4 *:Tecnologia* y 3 *:Curso*) y **26 relaciones** semánticas (*:DOMINA*, *:REQUIERE_TECNOLOGIA*, *:LIDERA*, *:POSTULA_A*, *:ENSENA_TECNOLOGIA*, *:TIENE_PRERREQUISITO*, *:SUPERVISA_CURSO* y *:DEPENDE_DE*).

Consultas Cypher Relacionadas con Necesidades del Proyecto y Recomendación

```
// CONSULTA 1: Indice de Compatibilidad Desarrollador - Proyecto
MATCH (p:Proyecto {convocatoriaAbierta: true})-[:REQUIERE_TECNOLOGIA]->(tReq:Tecnologia)
WITH p, collect(tReq) AS Stack, count(tReq) AS TotalReq
MATCH (d:Desarrollador) OPTIONAL MATCH (d)-[:DOMINA]->(tDom:Tecnologia) WHERE tDom IN Stack
WITH d, p, TotalReq, collect(tDom.nombre) AS Cumplidas, count(tDom) AS Cant WHERE Cant > 0
RETURN d.nombre AS Desarrollador, p.titulo AS Proyecto,
       round((toFloat(Cant)/TotalReq)*100.0, 1) AS PorcentajeCompatibilidad ORDER BY PorcentajeCompatibilidad DESC;

// CONSULTA 2 (RECOMENDACION DE CONOCIMIENTO IMPLICITO - CIERRE DE BRECHA CON CURSOS):
MATCH (d:Desarrollador {id: 'DEV-02'})-[:DOMINA]->(:Tecnologia)<-[:REQUIERE_TECNOLOGIA]-(p:Proyecto)
MATCH (p)-[:REQUIERE_TECNOLOGIA]->(tFaltante:Tecnologia) WHERE NOT (d)-[:DOMINA]->(tFaltante)
MATCH (c:Curso)-[:ENSENA_TECNOLOGIA]->(tFaltante) OPTIONAL MATCH (m:Mentor)-[:SUPERVISA_CURSO]->(c)
RETURN d.nombre AS Desarrollador, p.titulo AS ProyectoObjetivo, tFaltante.nombre AS HabilidadFaltante,
       c.titulo AS CursoRecomendado, c.horas AS Horas, m.nombre AS Mentor;

// CONSULTA 3 (INFERENCIA DE ASOCIACION IMPLICITA - DUPLAS COMPLEMENTARIAS FRONTEND + BACKEND):
MATCH (p:Proyecto {id: 'PROY-01'})
MATCH (d1:Desarrollador {perfilPrincipal: 'Frontend'})-[:DOMINA]->(t1:Tecnologia)<-[:REQUIERE_TECNOLOGIA]-(p)
MATCH (d2:Desarrollador {perfilPrincipal: 'Backend & Datos'})-[:DOMINA]->(t2:Tecnologia)<-[:REQUIERE_TECNOLOGIA]-(p)
RETURN p.titulo AS Proyecto, d1.nombre AS Frontend, collect(DISTINCT t1.nombre) AS AporteFront,
       d2.nombre AS BackendComplementario, collect(DISTINCT t2.nombre) AS AporteBack;
```

Consulta Cypher Ejecutada	Resultado Obtenido en Neo4j	Valor Semántico / Inferencia Generada
Consulta 1: Compatibilidad Desarrollador-Proyecto	Ana Sofía: 100% en Telemedicina; Mateo: 100% en FinTech y 66.7% en Telemedicina; Carlos Mario: 66.7% en Campus IoT y 33.3% en Telemedicina.	Automatiza la priorización de convocatorias según el dominio real del stack tecnológico.
Consulta 2: Recomendación de Cursos por Brecha (Skill Gap)	Para Carlos Mario en Telemedicina: detecta brecha en Node.js y PostgreSQL y recomienda 'Desarrollo de APIs REST con Node.js (32h)' y 'Modelado en PostgreSQL (28h)'.	Genera una recomendación personalizada no registrada directamente, uniendo brechas de un proyecto con la oferta formativa.
Consulta 3: Duplas Complementarias de Desarrollo	Empareja a Carlos Mario (aporta React.js) con Mateo Fernández (aporta Node.js y PostgreSQL) para el proyecto de Telemedicina.	Descubre una alianza implícita entre dos estudiantes sin vínculo previo que juntos logran el 100% del stack requerido.

4.9 Integración Coherente del Diseño

1. **Problema y Funcionalidades:** La desconexión entre estudiantes, proyectos y cursos se resuelve mediante el catálogo con cálculo de afinidad (F1), el diagnóstico de brechas (F2), la inferencia de rol (F3) y la recomendación de duplas complementarias (F5).
2. **Modelo Conceptual y Navegación:** Las clases centrales del dominio (*ProyectoSoftware*, *Desarrollador*, *HabilidadTecnica*, *CursoFormativo*) se traducen en índices y nodos navegacionales UWE dentro del prototipo interactivo.
3. **Ontología y Representación del Dominio:** La ontología *conectadev.owl* formaliza en Protégé los axiomas de equivalencia (*DesarrolladorFullStack*, *ProyectoListoParaIniciar*) y la transitividad de prerrequisitos, ejecutándose en el backend sin exponer complejidad técnica al usuario.
4. **Servicios Web y Conceptos del Modelo:** Los 6 endpoints REST y el contrato SOAP intercambian estructuras de datos alineadas 1 a 1 con las propiedades y clases del modelo conceptual.
5. **Grafo de Conocimiento e Inferencia:** Mientras OWL garantiza clasificación axiomática y consistencia formal, Neo4j recorre las conexiones multi-salto para recomendar cursos que cierran brechas de habilidades y conformar equipos complementarios.

6. **Soporte Arquitectónico Integral:** La arquitectura modular con persistencia políglota integra la interfaz web limpia, la capa de servicios REST/SOAP y los motores semánticos cumpliendo los atributos de usabilidad, mantenibilidad, seguridad y escalabilidad.

6. Planificación de Sustentación y Defensa Técnica (20 Minutos — Sección 6 y 7)

Bloque / Integrante	Tiempo	Secciones y Evidencias a Sustentar	Dimensión Rúbrica (SABER / SER / HACER)
Integrante 1	00:00 - 05:00	Formulación del problema (4.1), Fundamentación académica (4.2) y Metodología UWE (4.4).	SABER y SER: Comprensión de la necesidad web y justificación metodológica.
Integrante 2	05:00 - 10:00	Arquitectura Web (4.3), Modelo Conceptual y Navegacional, Prototipo y Heurísticas de Nielsen (4.5).	SABER y SER: Diseño centrado en el usuario y atributos de calidad arquitectónicos.
Integrante 3	10:00 - 15:00	Servicios Web REST y SOAP/WSDL (4.6) y Ontología OWL con razonador HermiT en Protégé (4.7).	SABER y HACER: Construcción ontológica, restricciones, preguntas de competencia e inferencias.
Integrante 4	15:00 - 20:00	Grafo de Conocimiento en Neo4j, Consultas Cypher de recomendación (4.8) e Integración Total (4.9).	HACER y SER: Consultas en grafo, conocimiento implícito e integración extremo a extremo.